

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

DM576: DATABASE SYSTEMS

Takehome Exam

AUTHOR:

Jesper Roager

Jeroa25

June 1, 2026



Contents

1	ER to Relation model	2
1.1	E/R	2
1.2	ER To Relation	3
1.2.1	Object-oriented	4
1.2.2	Use Nulls	4
1.2.3	E/R - style	4
1.3	Add atriubte to player	4
2	SQL queries	5
2.1	Query A	5
2.2	Query B	5
2.3	Query C	6
2.4	Query D	6
2.5	Query E	6
3	Relational algebra	6
3.1	Question A	6
3.2	Question B	6
4	Normalization	6
4.1	Question A	6
4.2	Question B	6
4.3	Question C	7
4.4	Question D	7
4.5	Question E	7
5	Indices	7
5.1	Question A	7
5.2	Question B	7
5.3	Question C	7
5.4	Question D	7

1 ER to Relation model

1.1 E/R

I start with adding the team to the ER-diagram. It is a simple entity set with three attributes. Then I have to consider which is the primary key. I choose **Name** because the database is for the Danish football league, which is not that therefor there can only be one team with the same name.

Then we have the Coaches and Owners, they both have the same three attributes, (**CPRNo**, **Name**, **Address**). I make an entity set called **Person**, with these attributes and the primary key is **CPRNo** because it will always be unique by definition. To model a Coach I just say it **IsA Person** with the extra attribute, **Years of experience**. A coach maybe coaches a team, if they do the only coach one team. This is indicated by the filled arrow the relation **Coaches** to the team entity set. The relation **Coaches** has an attribute to tell which type of coach he/she is.

The owner is also a Person, so it is model with a **IsA** relation from **Owner** to **Person**. An owner has in addition a net worth, which I have add as an attribute to the Owner. Much in the same ways as a coach, can a owner maybe own a part of a team. And if they do, the relation **owns** has an attribute note how much he/she owns of the team.

Next up I add **Player**, which i have decided not to model as a person, because it does not have the attribute **CPRNo**. Instead, I have made it a weak entity set, because the **PlayerNumber** is only unique with in the team and the name not even necessarily unique with in the team. Therefore, is it weak, and it uses **PlaysOn** as the supporting relation to identify a player uniquely, with the attribute **Number**, which is the primary key of **Player**. **PlaysOn** is a one-to-many relationship, so it can be used for this porpus. The curved end of this relationship ensures that a player is always is in on a Team. This is necessary to use it for a weak entity. The rest attributes are simply added.

Lastly, I add the Game entity set. I have decided to model the **HomeTeam** and **AwayTeam** as relation. Game is a weak enity set because we need the two teams to identify which game has been played. Therefore it is weak with two supporting relations, **Home** and **Away**. But if i only had this as the key in Game, it would not be possible for two teams to play each other more than once. A solution to this is adding the time of the game, which I have as a primary key in **Game**. It also has score attribute.

See ER-diagram in figure 2:

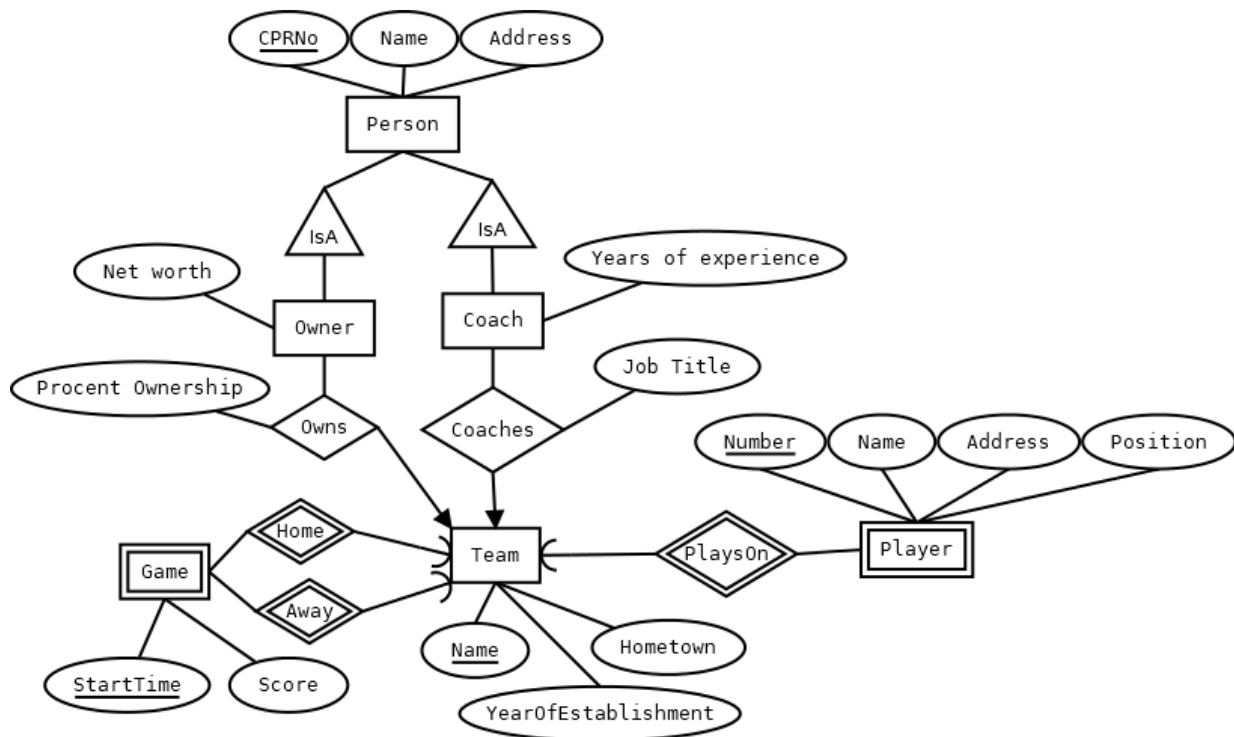


Figure 1: My ER

1.2 ER To Relation

First, I make the team into a schema because there are lots of things reffectering it, but it does not refence anything. It is the ecate same as the ER, but just in schema instead, same attributes, same key. See below:

Team(Name, Year Of Establishment, Hometown)

Then we have the player, we pull the primary key of **Team** into this schema and call it **TeamName**. It indicates which the player plays on. When it is a one to many relation we can contian all the information on the one side, so by this we have included all the information in the **PlaysOn** relation in this schema. The schema also have all the attributes from the ER diagram, the **PlayerNumber** still being a primary key. See below:

Player(TeamName, Number, Name, Address, Position)

With the Game schema I do much the same thing as above. I pull the primary key from **Team** down twice, but call it something different, to not have duplicate attribute names. And again keep the key from the diagram, and add the last attribute. Then it looks like this:

Game(HomeTeam, AwayTeam, StartTime, Score)

The last schema, can look a bit different depding which style of Subclasses I use. I will show all different styles

1.2.1 Object-oriented

In object-oriented style all instance of a subclass gets their own relation with all the relevant data. I have also put the relationship **Owns** and **Coaches** in their respective relations, with their attribute. (**WorksOnTeam** and **JobTitle** in **Coach**, **OwnsTeam** and **Procent Ownership** in **Owner**) This will be null if the owner does not own a team or a coach that does not coach. Because the relation is the own/coach zero or one team, does not have anything to do with style of subclassing.

```
Coach(CPRNo, Name, Address,
      Years Of Experince, WorksOnTeam, JobTitle)
Owner(CPRNo, Name, Address,
      NetWorth, OwnsTeam, Procent Ownership)
```

1.2.2 Use Nulls

In null style we have on big relation, where the attributes that are not relevant for a specific entry are null. Which results in this big schema, still with the relationship between owners and teams and the relationship between coaches and teams embed:

```
Person(CPRNo, Name, Address, Years Of Experince, NetWorth,
      OwnsTeam, Prcent Ownership,
      WorksOnTeam, JobTitle)
```

1.2.3 E/R - style

In E/R style, it refencers to the superclass with the key, and then only keep the revalnt data in the subclasses. So I add a relation for a person then **Coach** and **Owner** references it with the primary key. Still the relationship reffecering them to the team is embed in the relation.

```
Person(CPRNo, Name, Address)
Coach(CPRNo, Years Of Experince, Team, JobTitle)
Owner(CPRNo, NetWorth, Team, Procent Ownership)
```

1.3 Add atriubte to player

This would make it possible for the player to be subclassed of a person. And therefor it would make so it is not weak anymore. The number atriubte can be moved to the relationship **PlaysOn**, where it makes must sense, because it is tied to a player on a specific team not the player. Cause if the player moves teams he/she might get a new number. The postion a player plays does not change when change teams, so there for it is an attribute

on the player itself. (I assume this, i have no knowledge of football) The new ER diagram looks like this:

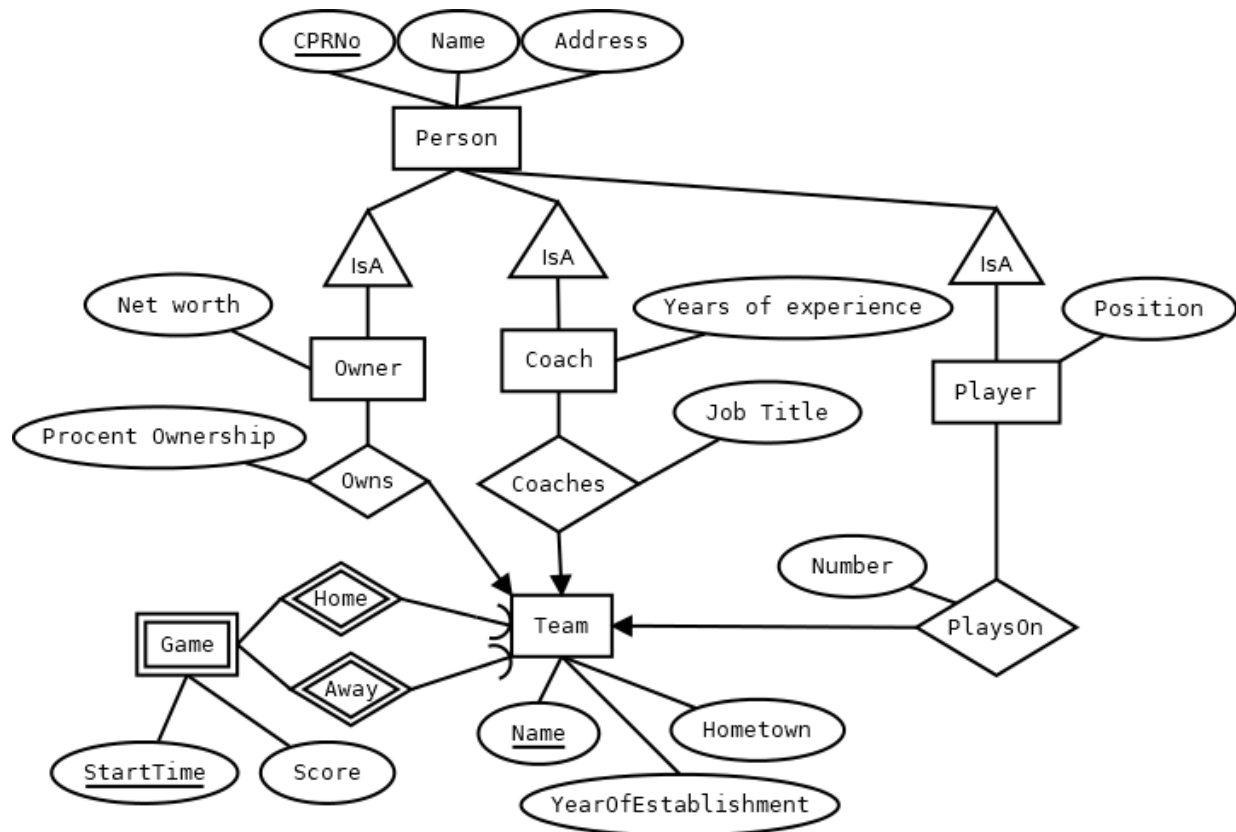


Figure 2: My ER with player having a CPRNo

The only schema that changes is the for player, it loses the attributes that are in the person schema. And get a new primary key. Teamname is no longer a primary key, because the entity set is not weak anymore. It ends up looking like this:

Player(CPRNo, TeamName, Number, Position)

2 SQL queries

2.1 Query A

The correct is (iii).

2.2 Query B

These are correct (ii) and (iv).

2.3 Query C

These are correct (iii) and (iv).

2.4 Query D

These are correct (ii), (iii) and (iv).

2.5 Query E

These are correct (i), (ii) and (v).

(ii) is correct in the example data set, but if there is added a membership level then it does the wrong thing

3 Relational algebra

3.1 Question A

1. A
2. C
3. B
4. D
5. A

3.2 Question B

1. D
2. C

4 Normalization

4.1 Question A

The candidate key (ii)

4.2 Question B

(iii) is the set of all non-trivial FDs that follow from the given FDs.

4.3 Question C

(iii) is a minimal basis

4.4 Question D

Is correct (iv).

4.5 Question E

Is correct (ii).

5 Indices

5.1 Question A

(i) and (iii)

5.2 Question B

(i) and (iii)

5.3 Question C

(i) and (iii)

5.4 Question D

(i), (iv) and (v)